

Einführung in die Programmierung

Prof. Dr. Gemma Roig
Prof. Dr. Matthias Kaschube
Dr. Karsten Tolle



Übungsblatt ÜE-04

Ausgabe: 12.11.2025
Abgabe: 22.11.2025
12:00 Uhr

Schatzsucher-Spiel

Hinweis:

- Die Abgabe bringt keine Bonuspunkte. Ihre Tutorin/Tutor erstellt Ihnen ein kurzes Feedback zur Abgabe. Das ganze dient also als Übung und Lernerfolgskontrolle für Sie!
- Sie dürfen für dieses Blatt alles nutzen, jedoch ist es nicht zielführend automatisch generierte Lösungen (z.B. über ein LLM) einfach zu kopieren und hochzuladen. **Bitte aber die Quellen angeben, also auch ob ein LLM bei der Erstellung unterstützt hat!**
- Geben Sie am Anfang aller Ihrer Quellcode Dateien (.py) die Variable `__author__` an (am Anfang des Quellcodes) – ist schon mal eine Übung, da dies für die Codereview-Aufgaben, welche für die Studienleistung relevant sind wichtig sein wird:
`__author__ = "<Matr-Nr>, <Nachname>"` also z.B.:
`__author__ = "1234567, Tolle"`
- Digitale Abgaben, die nicht im Format **.pdf** oder **.txt** für Texte oder **.py** für Code erfolgen, werden nicht überprüft. Bei Abgaben mehrerer Dateien müssen diese als **.zip** zusammengefasst werden.
- Immer wenn wir von Python Programm sprechen, meinen wir Python Code, der mindestens mit der Python Version 3.10 lauffähig ist. Bitte aktuell komplett ohne Imports arbeiten.
- Sollten Aufgabenstellungen nicht eindeutig sein, bitte durch Ihre eigenen Annahmen ergänzen und diese dokumentieren (z.B. in einem Kommentar).
- Achten Sie auch auf Vorgaben Ihrer Tutorin/Tutors. Hierfür auch in das Gruppenforum im Moodle-Kurs von EPR schauen!

Aktualisierung der Hinweise:

- Beim Programmieren und Kommentieren halten Sie sich bitte an die Regeln im Programmierhandbuch (Style Guide), siehe Moodle-Kurs. Im Zweifelsfall gilt PEP 8. ... möglichst selbst mit `pycodestyle` mal prüfen.

Aufgabe ÜE-04-1: Schatzsucher-Spiel

In dieser Aufgabe soll ein Konsolenspiel in Python implementiert werden, bei dem der Spieler in einer 5x5 Karte nach einem Schatz sucht. Der Spieler beginnt oben links und hat eine begrenzte Anzahl an Zügen (10 Züge), um den Schatz zu finden. Auf der Karte gibt es Hindernisse, die der Spieler nicht betreten kann. Ziel ist es, den Schatz zu finden, bevor die Züge aufgebraucht sind.

Vorgaben:

- **Spielfeld:**
 - Erstellen Sie ein 5x5 Gitter als Spielfeld.
 - Standardmäßig sind alle Felder leer (".").
 - Der Schatz ("X") und Hindernisse ("O") werden zufällig platziert.
 - Die Position des Spielers wird mit "P" markiert.
- **Spielmechanik:**
 - Der Spieler gibt Befehle ein ("o" für oben, "u" für unten, "l" für links, "r" für rechts), um sich zu bewegen.
 - Der Spieler darf nicht außerhalb der Karte ziehen und nicht auf Felder mit Hindernissen gelangen.
 - Nach jedem Zug wird die Karte aktualisiert und angezeigt.
 - Das Spiel endet, wenn der Spieler den Schatz findet oder die Züge aufgebraucht sind.
 - Optional kann die Entfernung zum Schatz bei optimaler Schrittreihenfolge angezeigt werden, um die Suche spannender zu gestalten.
- **Abbruch des Spiels:**
 - Der Spieler kann das Spiel jederzeit mit der Eingabe eines bestimmten Befehls ("q") abbrechen.

Funktionen:

1. **create_map() (1,5 Punkte):**
 - **Beschreibung:** Erstellt das 5x5 Spielfeld und initialisiert es mit leeren Feldern (".").
 - **Parameter:** Keine.
 - **Rückgabewert:** Eine Liste von Listen (5x5 Gitter).
2. **place_element(game_map, element) (1,5 Punkte):**
 - **Beschreibung:** Platziert ein Element (z. B. Schatz "X" oder Hindernisse "O") zufällig auf der Karte. Die Platzierung erfolgt nur auf freien Feldern.
 - Hindernisse sollen versteckt bleiben, bis der Spieler auf eins stößt. Danach werden sie auf der Karte angezeigt, damit der Spieler nicht erneut darauf stößt.
 - **Parameter:**
 - game_map: Das Spielfeld (Liste von Listen).
 - element: Das zu platzierende Element (String).
 - **Rückgabewert:** Ein Tupel (x, y) mit den Koordinaten des platzierten Elements.
3. **print_map(game_map) (1 Punkt):**
 - **Beschreibung:** Gibt die aktuelle Karte in der Konsole aus. Jede Zeile wird auf einer neuen Zeile angezeigt, und Felder werden durch Leerzeichen getrennt.
 - **Parameter:**
 - game_map: Das Spielfeld (Liste von Listen).
 - **Rückgabewert:** Keinen.



4. **move_player(position, direction, game_map, obstacles) (2 Punkte):**
 - **Beschreibung:** Berechnet die neue Position des Spielers basierend auf der eingegebenen Richtung. Die Funktion verhindert Bewegungen außerhalb der Karte oder auf Hindernisse. Bei ungültigen Zügen bleibt der Spieler an seiner aktuellen Position.
 - Optional: Ungültige Züge reduzieren nicht die Zuganzahl. (siehe Hinweise zur Dokumentation)
 - **Parameter:**
 - position: Aktuelle Position des Spielers (x, y) (Tupel).
 - direction: Richtung ("o", "u", "l", "r") (String).
 - game_map: Spielfeld (Liste von Listen).
 - obstacles: Liste mit allen Positionen der Hindernisse (Tupel).
 - **Rückgabewert:** Ein Tupel (x, y) mit den neuen Koordinaten des Spielers.
5. **treasure_hunter_game() (4 Punkte):**
 - **Beschreibung:** Steuert das gesamte Spiel:
 - Initialisiert das Spielfeld und platziert den Schatz und die Hindernisse.
 - Verarbeitet die Eingaben des Spielers und bewegt ihn entsprechend.
 - Zeigt nach jedem Zug die aktualisierte Karte an.
 - Optional: Zeigt die Entfernung zum Schatz an.
 - Beendet das Spiel, wenn der Schatz gefunden wurde oder die Züge aufgebraucht sind.
 - **Parameter:** Keine.
 - **Rückgabewert:** Keinen.

Hinweise zur Dokumentation:

Nutzen Sie Ihre Dokumentation und Docstrings, um Details zu den Regeln und Funktionen klar zu formulieren. Beispiele:

- Wird die Zuganzahl bei ungültigen Zügen reduziert oder nicht?
- Werden Hilfestellungen wie die Entfernung zum Schatz angezeigt? (Im Fall, dass, der Schatz versteckt ist)
- Der Schatz kann sichtbar sein, oder die Suche fokussiert sich auf das Finden des Weges.

Das Spiel kann nach Belieben erweitert werden.

Die Funktionen 1-4 sollten in einem eigenen Modul implementiert werden. Funktion 5 in einem anderen Modul.

Vergessen Sie bitte nicht, die Funktionen 1-4 unter `if __name__ == "__main__"` zu testen.